

VMS: Visitors Management System

Final Project Report

Module: CS107.3 - Object Oriented Programming with C#

Module Leader: Anton Jayakody

Group: C

Institution: NSBM Green University

GitHub Link: <https://github.com/SAYR622/VMS.git>

1. Team Composition

1. S A Y Randhinu - Student ID: 38677
2. K T Damhiru - Student ID: 38035
3. W A A S H Weerasooriya - Student ID: 37910
4. T O H Jinasinghe - Student ID: 38440
5. H N B Hansana - Student ID: 38712
6. W G C Isidhara - Student ID: 38641
7. P D Weerathunga - Student ID: 37514
8. N Nimjaya - Student ID: 38294
9. P M L Hewawasam - Student ID: 38449

2. Problem Statement

Existing Problem

Sri Lanka is home to globally significant cultural heritage sites such as Jaya Sri Maha Bodhi, Ruwanweli Maha Seya, and Kataragama which attract millions of local pilgrims and international tourists annually. Despite this massive footfall, the administrative infrastructure managing these sites remains largely manual, fragmented, or non-existent. Site administrators lack real-time visibility into visitor statistics, making it difficult to monitor capacity or track historical trends.

More critically, there is a distinct communication void. During emergencies, unexpected site closures, maintenance operations, or even for general historical dissemination, administrators have no direct, reliable channel to communicate with the people currently occupying the site. Traditional PA systems are localized, and physical notice boards are easily overlooked.

Contribution as a Solution

The Visitors Management System (VMS) bridges this administrative gap by introducing a hybrid distributed software solution. It modernizes site management without imposing heavy technological barriers on the end-user. VMS provides administrators with a centralized, secure desktop application to visualize real-time data, manage administrative roles, and orchestrate communication. Simultaneously, it offers a frictionless, zero-install mobile web portal for visitors to register their presence. By securely collecting visitor emails via location-specific QR codes, VMS establishes a direct communication pipeline. This empowers administrators to deploy automated informational templates or custom emergency broadcast campaigns via a cloud-based infrastructure, fundamentally modernizing heritage site administration while strictly maintaining data privacy.

3. Requirements

Target Audience

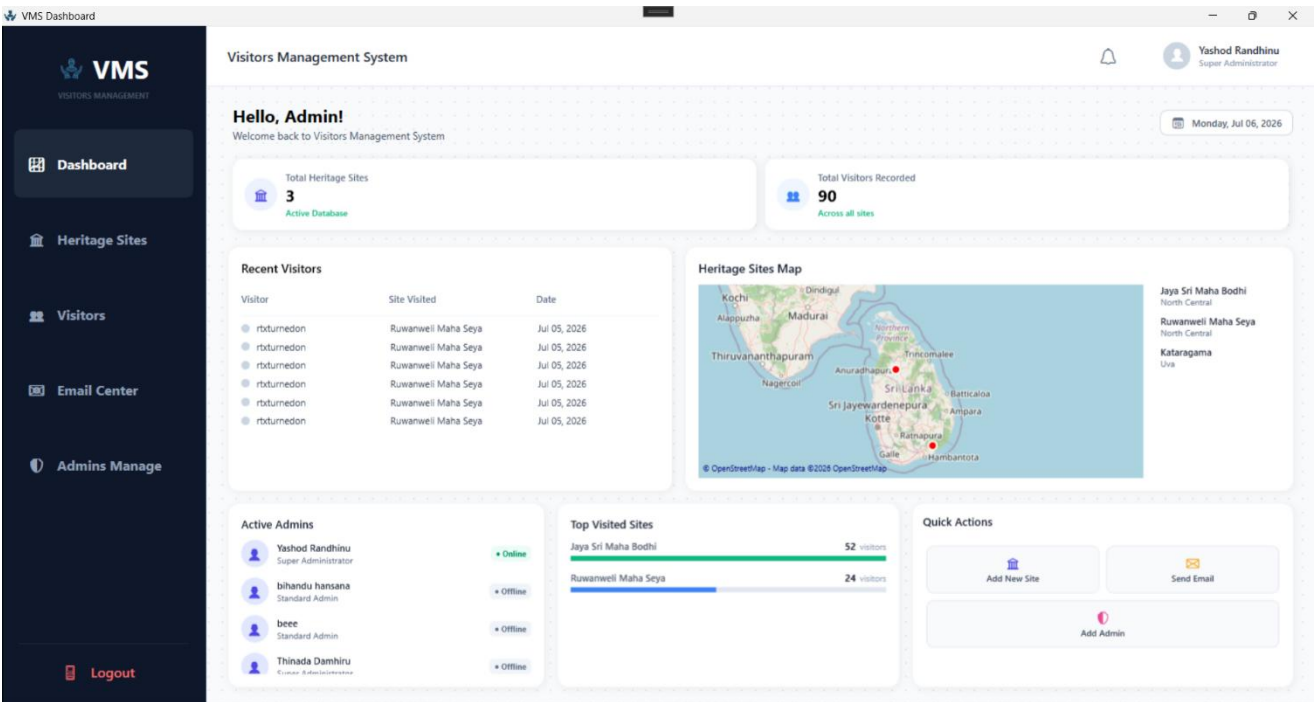
VMS is designed for two distinct user bases. The primary audience consists of **Heritage Site Administrators**. This includes "Super Administrators" who oversee the entire system matrix and "Standard Admins" who manage day-to-day communications and site-specific data. The secondary audience comprises **Visitors and Tourists**. This demographic requires a low-friction interface that respects their time and privacy, as they are unlikely to download a dedicated mobile application for a brief site visit.

Included Features and Rationale

1. **QR-Triggered Web Portal with Optimistic UI:** Visitors scan a physical QR code to access a mobile web form. To accommodate visitors' impatience and prevent premature tab closures, an "Optimistic UI" was implemented via JavaScript. It instantly renders a success screen upon submission while silently processing database insertions and edge function triggers in the background.



2. **WPF Desktop Dashboard with Spatial Mapping:** The core administrative hub was built using C# WPF. It includes real-time statistical aggregations and integrates GMap.NET to plot active sites geographically, providing administrators with immediate spatial awareness of the system's operational scope.



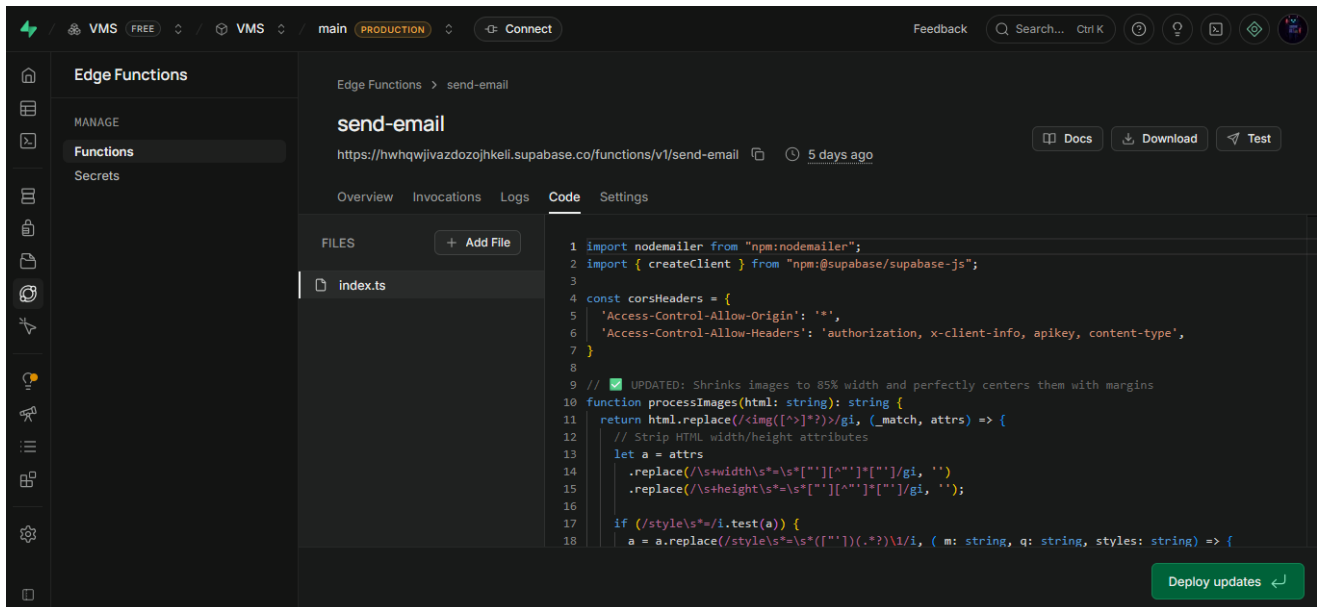
1. Map in Dashboard

3. **Custom Email Engine (WYSIWYG & Edge Functions):** Administrators require the ability to send rich, formatted announcements. A Quill.js WYSIWYG editor was embedded into the WPF client, allowing admins to design emails. The sending process is outsourced to Supabase Edge Functions (TypeScript), which wrap the raw HTML in responsive, branded templates and dispatch them via SMTP, ensuring reliable delivery without burdening the client application.

The screenshot shows the Supabase Table Editor interface. The 'public.admins' table is selected. The table has columns: adminID, created_at, status, a_type, username, password, and name. There are four rows of data. The interface includes a sidebar with a list of tables (admin_req, admins, email_templates, sites, visitors_emails) and a top bar with search and navigation options.

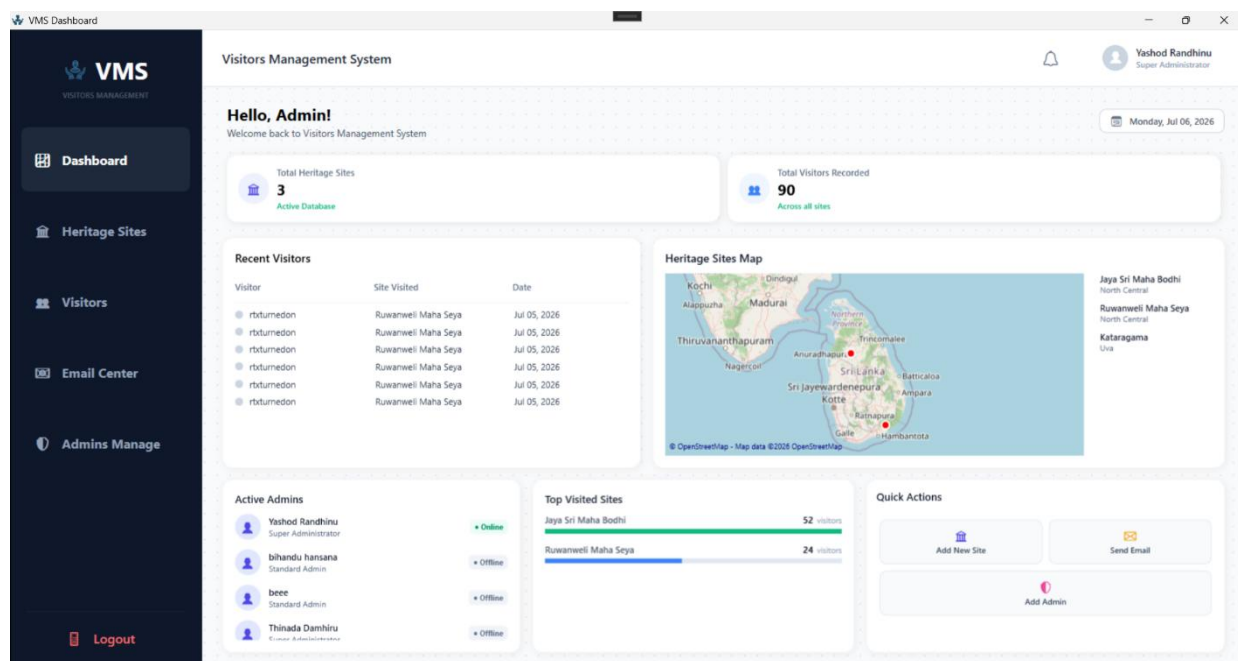
adminID	created_at	status	a_type	username	password	name
1	2026-06-18 17:48:32.85858+	TRUE	super	sayr	sayr622	Yashod Ranc
2	2026-06-27 22:23:26.780132	FALSE	super	thina	thina2005	Thinada Dan
3	2026-06-27 22:39:09.761724	FALSE	standard	chanux	chanux	beee
6	2026-07-04 10:47:59.90645	FALSE	standard	Beeeee	bee1234	bihandu han

2. Tables in Supabase

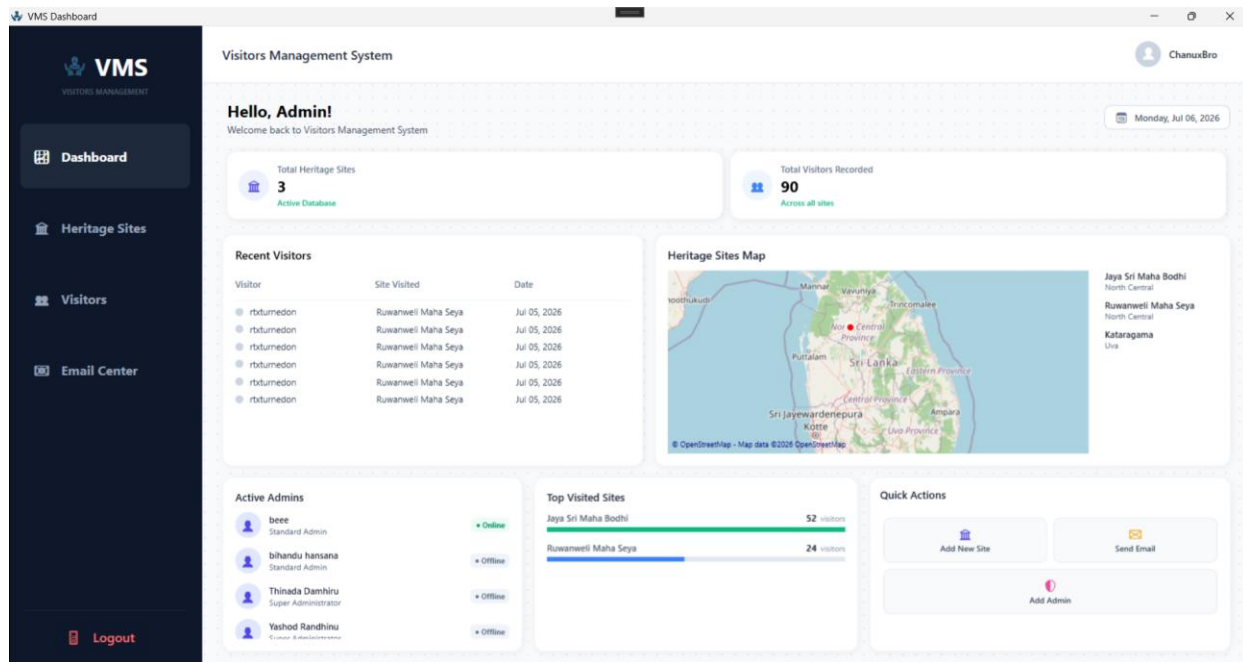


3. The Edge Function

- Role-Based Access Control (RBAC):** To ensure system security, strict role-based routing was implemented, limiting sensitive actions (like adding new admins) strictly to Super Administrators.



4. A Super Admin



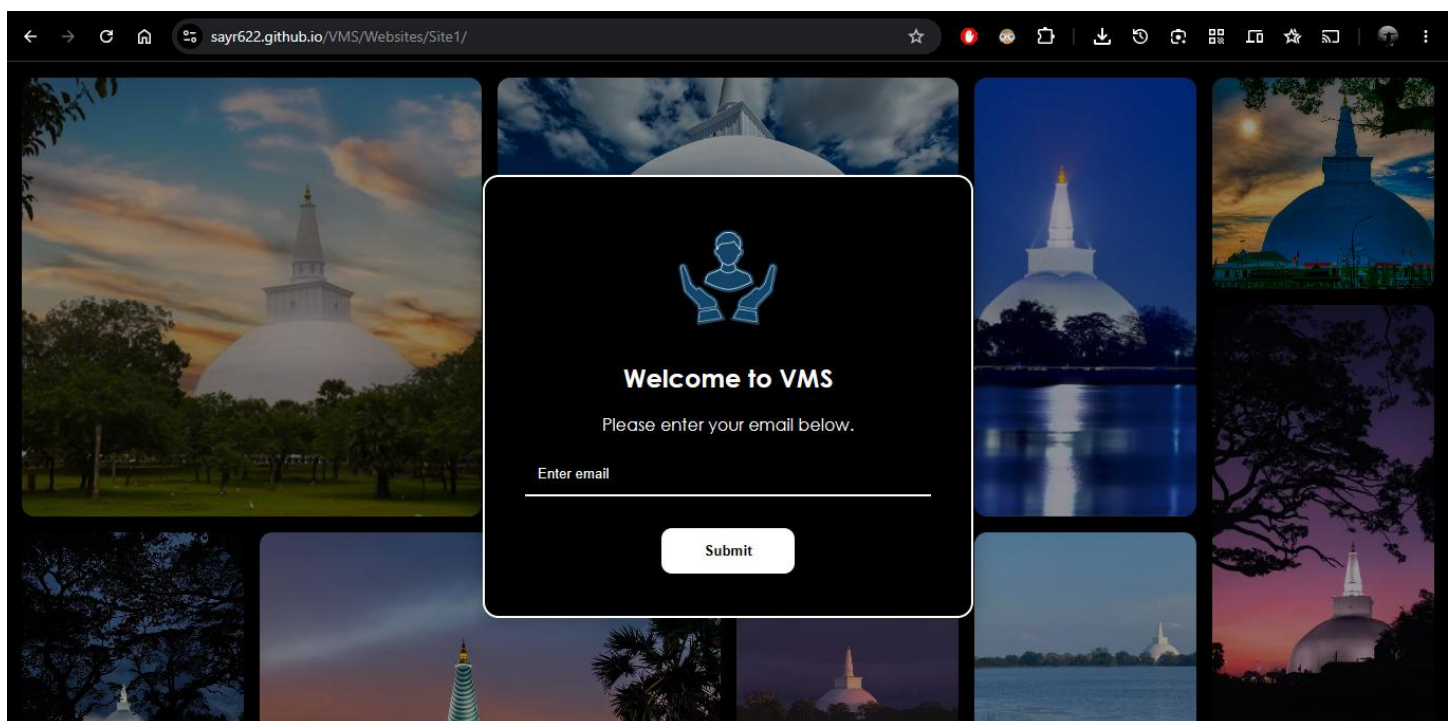
5. A Standard Admin

4. Design

System Architecture

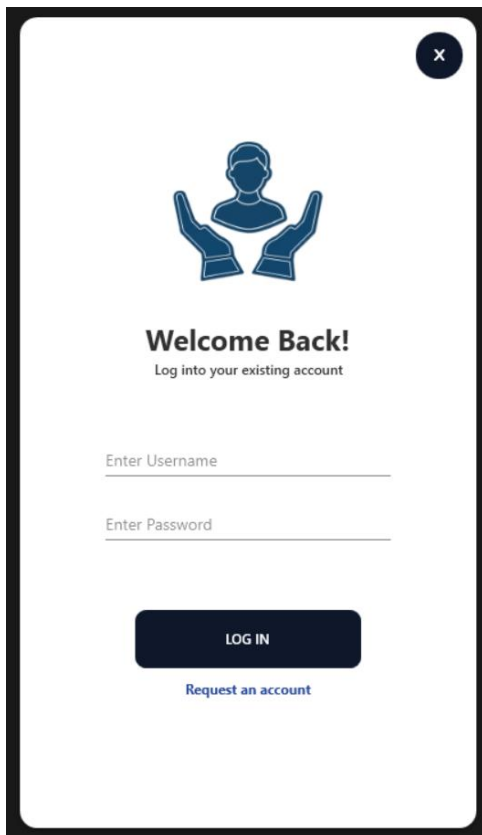
VMS utilizes a modern Client-Server architecture heavily reliant on Cloud Infrastructure. The system comprises a rich C# WPF Desktop Client for administrators and a lightweight HTML/JS web client for visitors. Both clients interface seamlessly with a centralized Supabase backend, which provisions a PostgreSQL database for relational data storage, Supabase Storage for image hosting, and Edge Functions for serverless compute operations.

6. Web Client for the Site1

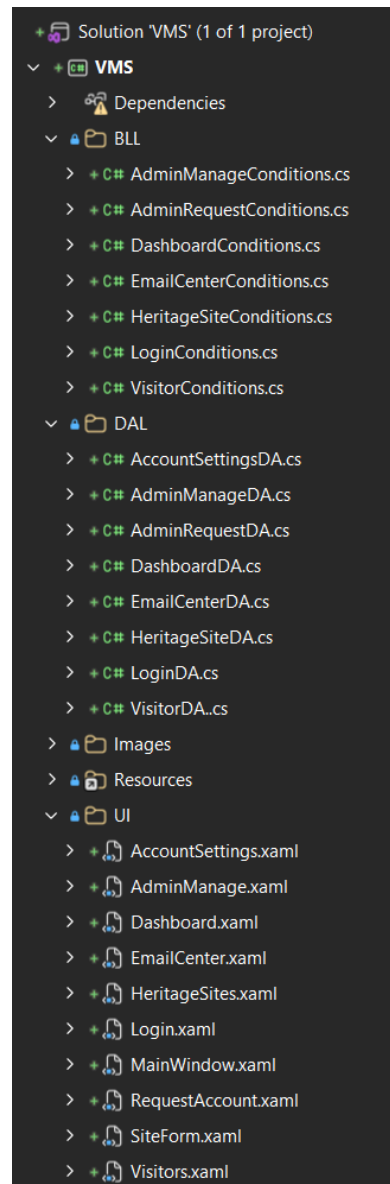


Process Interaction

The system's processes are highly decoupled. For visitor registration, the mobile web client captures the email, executes an INSERT operation to the Supabase database, and simultaneously makes a POST request to the Supabase Edge Function. The Edge Function then independently handles email compilation and SMTP dispatch. On the administrative side, when an admin logs in, the UI captures the credentials, passes them through the Business Logic Layer (BLL) for validation, and forwards them to the Data Access Layer (DAL). The DAL queries the PostgreSQL database and returns the authorization token and user role back up the chain to render the appropriate dashboard UI.



7. Log In Page for the Admin

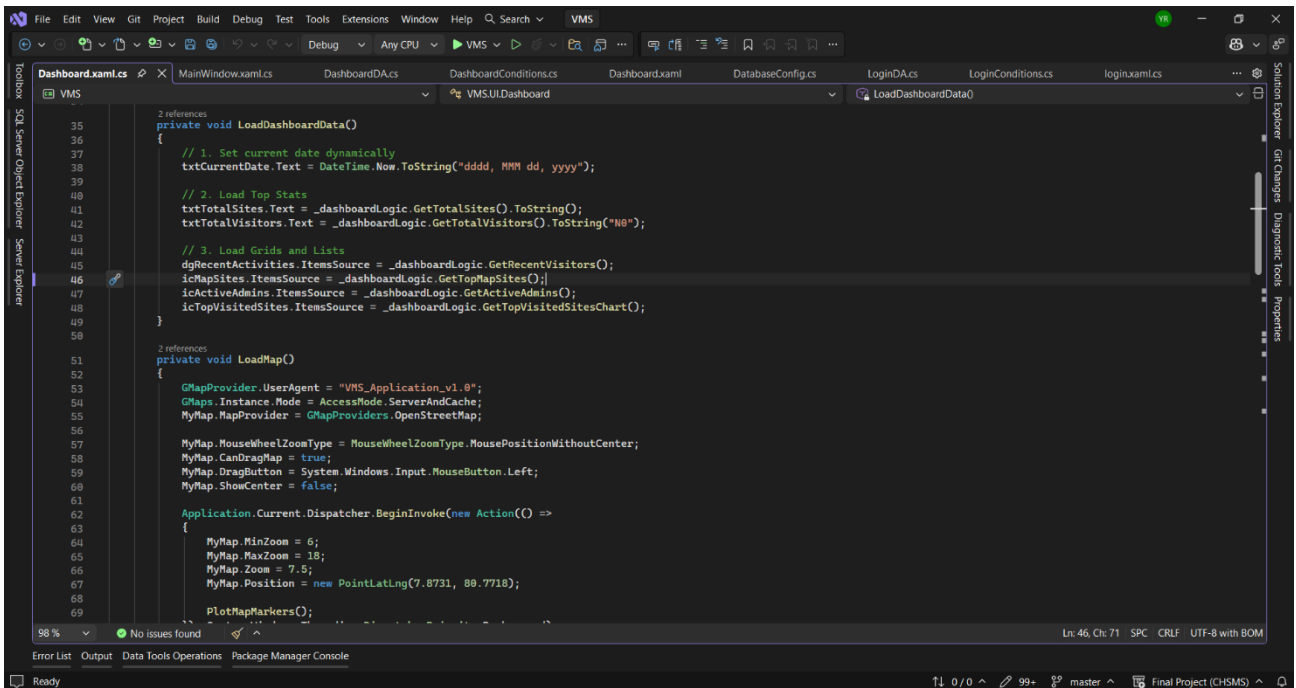


8. UI, BLL & DAL Folders

Data and Code Structure

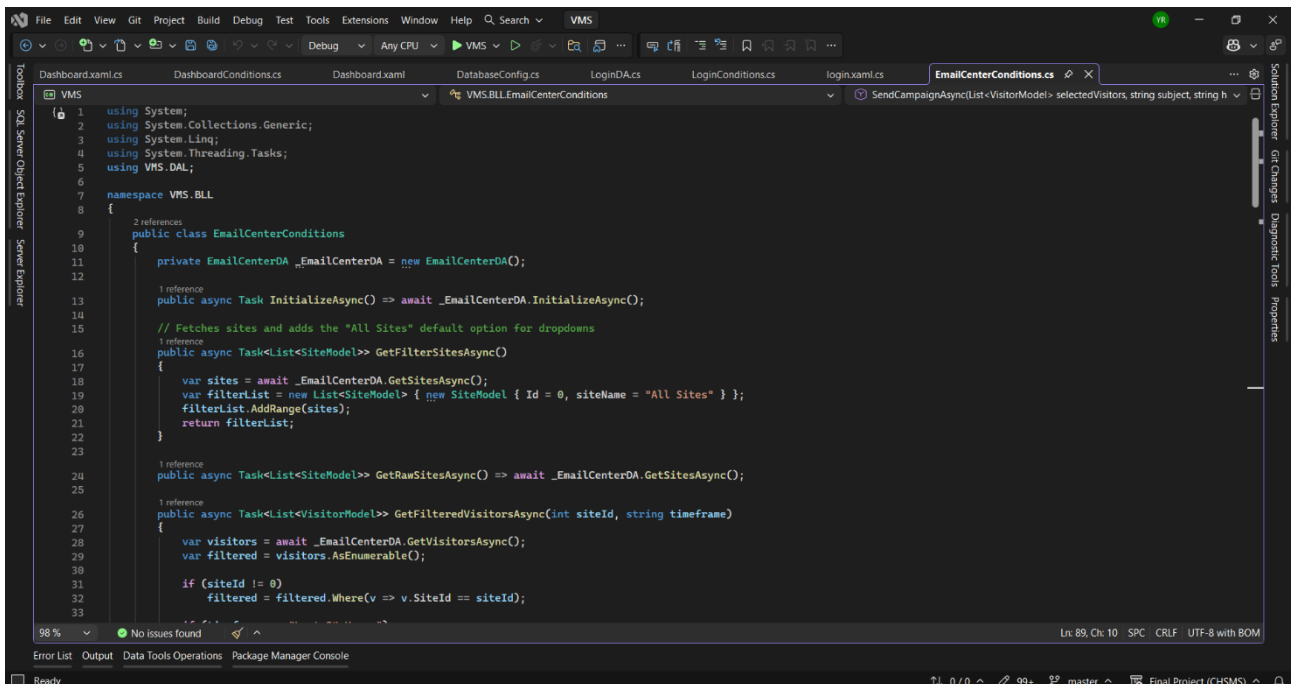
The desktop application strictly adheres to a **Three-Tier Architecture**:

- **Presentation Layer (UI):** Consists of XAML files and code-behind (e.g., Dashboard.xaml.cs). It is strictly responsible for rendering data and capturing input.



9. Dashboard.xaml.cs

- **Business Logic Layer (BLL):** Consists of condition classes (e.g., EmailCenterConditions.cs). It enforces business rules, validates inputs, transforms raw database data into viewable formats, and catches errors before they trigger expensive database calls.



10. EmailCenterConditions.cs

-
- The screenshot displays the Visual Studio Code interface with the following components:
- File Explorer (Left):** Shows the project structure with folders like 'Toolbox', 'SQL Server Object Explorer', and 'Server Explorer'.
 - Code Editor (Center):** Displays the file 'VMS.DALVisitorModel.cs'. The code includes:


```

1  using System;
2  using System.Collections.Generic;
3  using System.Threading.Tasks;
4  using Supabase;
5
6  namespace VMS.DAL
7  {
8      2 references
      public class EmailCenterDA
9      {
10         private Client _supabase;
11
12         // Initializes the Supabase C# Client specifically for Storage & Edge Functions
13         1 reference
14         public async Task InitializeAsync()
15         {
16             if (_supabase != null) return;
17
18             string url = "https://mhqqsjvaxdozozjhkell.supabase.co";
19             string key =
20                 "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3ZHMzIjoiZjZlZDhYbWZlZSIsInJlZlI6Imh3aHF3amZlYXpkb3pvaHRZbWp1Iiwicm9sZSI6ImNlcnZpY2Vfc9sZSIsImhldCI6MTc4MTc1ODA2IiwiaWZyIjoiYm9kZM3IjzH9MDY3fQ.8BrcFvJ8Hz9HROHl6kQA8L23zItj6SsoF8IzFQBUrXk";
21
22             var options = new SupabaseOptions { AutoConnectRealtime = false };
23             _supabase = new Client(url, key, options);
24             await _supabase.InitializeAsync();
25         }
26
27         // --- READ OPERATIONS ---
28         2 references
29         public async Task<List<SiteModel>> GetSitesAsync()
30         {
31             var response = await _supabase.From<SiteModel>().Get();
32             return response.Models;
33         }
34
35         1 reference
36         public async Task<List<VisitorModel>> GetVisitorsAsync()

```
 - Solution Explorer (Right):** Shows the project structure with folders like 'Solution Explorer', 'Get Changes', 'Diagnostic Tools', and 'Properties'.
 - Status Bar (Bottom):** Shows the current file is 'Ln: 114, Ch: 36' and the encoding is 'UTF-8 with BOM'.

```

    graph TD
        subgraph "Data Access & Cloud"
            A[Invoke Edge Function] --> B[Send via SMTP]
        end
        subgraph "Admin Desktop UI"
            Start(( )) --> C[Select Visitors]
            C --> D[Write Email]
            D --> E[Click Send]
            E --> F[Show Success]
            F --> End(( ))
            G[Show Error Message] --> C
        end
        subgraph "VMS Business Logic Layer"
            H{Is Valid?} -- Yes --> I[Filter Distinct Emails]
        end
        AdminDesktopUI --> DataAccessCloud
        AdminDesktopUI --> VMSBusinessLogicLayer
        VMSBusinessLogicLayer -- No --> AdminDesktopUI
  
```

The diagram illustrates the Email Marketing Process, which is divided into three main layers: Data Access & Cloud, Admin Desktop UI, and VMS Business Logic Layer.

Data Access & Cloud Layer: This layer contains the process of invoking the Edge Function and sending the data via SMTP. The flow is: Invoke Edge Function → Send via SMTP.

Admin Desktop UI Layer: This layer represents the user interface. The process starts with a start node (black circle) leading to Select Visitors. From Select Visitors, the flow goes to Write Email, then Click Send, then Show Success, and finally to an end node (black circle). A Show Error Message box also leads to Select Visitors.

VMS Business Logic Layer: This layer contains the business logic. It starts with a decision diamond "Is Valid?". If the answer is "Yes", the flow goes to "Filter Distinct Emails". If the answer is "No", the flow loops back to the Admin Desktop UI layer.

Flow Summary: The process begins in the Admin Desktop UI with "Select Visitors". This triggers a call to the "Data Access & Cloud" layer to "Invoke Edge Function" and "Send via SMTP". The flow then returns to the Admin Desktop UI to "Write Email" and "Click Send". The "VMS Business Logic Layer" checks if the data is "Valid". If not, it loops back to the Admin Desktop UI. If valid, it proceeds to "Filter Distinct Emails" before returning to the Admin Desktop UI to "Show Success".

```
graph TD; Start(( )) --> Scan[Scan QR Code]; Scan --> Input[Input Email]; Input --> Submit[Click Submit]; Submit --> Success[Show Success Screen]; Success --> End(( )); Success <--> VMS[VMS Database]; VMS --> SEF[Supabase Edge Functions]; SEF --> Trigger[Trigger Email Function]; Trigger --> Dispatch[Dispatch HTML Email];
```

The flowchart illustrates the process flow for the Mobile Web Portal. It begins with a start node (black circle with a white dot) leading to a rectangular box labeled "Scan QR Code". This is followed by an arrow pointing to another rectangular box labeled "Input Email", which then points to a third rectangular box labeled "Click Submit". From "Click Submit", an arrow points down to a rectangular box labeled "Show Success Screen". From "Show Success Screen", an arrow points down to a black square connector. This connector has a curved arrow pointing up back to "Show Success Screen" and a straight arrow pointing down to a rectangular box labeled "VMS Database". Inside the "VMS Database" box is a smaller rectangular box labeled "Insert Email Record". An arrow points down from the "VMS Database" box to a larger rectangular box labeled "Supabase Edge Functions". Inside this box are two smaller rectangular boxes: "Trigger Email Function" on the left and "Dispatch HTML Email" on the right, connected by an arrow pointing from left to right.

11. Visitor Registration

Appropriateness of Structures

The Three-Tier Architecture was chosen for its strict separation of concerns. By isolating the DAL, we were able to implement parameterized SQL queries uniformly, effectively neutralizing SQL injection vulnerabilities. Placing validation inside the BLL ensures defensive programming, reducing unnecessary network latency to the cloud database. The utilization of Supabase Edge Functions for email dispatch was appropriate as it offloaded heavy, synchronous network tasks from the UI thread, preventing the desktop app from freezing during large campaign broadcasts.

5. Personal Reflection

S A Y Randhinu - Building the login, dashboard, and admin management modules taught me the real value of organizing code into separate layers. It made fixing bugs much easier because the visual design and the database rules were kept completely apart.

K T Damhiru - Suggesting we move our database to the cloud with Supabase was a huge turning point for the project, and it really taught me the value of modern cloud tools. Building the email center was a tough challenge, especially figuring out how to embed a web-based text editor inside a Windows desktop app and getting C# and JavaScript to talk to each other.

W A A S H Weerasooriya - Working on the visitors page taught me how to turn raw database numbers into useful visual charts. One big challenge was getting the bar chart text to fit perfectly without overlapping, which showed me that making a software interface look professional takes careful tweaking.

T O H Jinasinghe - Working on the heritage sites page taught me a lot about integrating interactive maps into a desktop application. A big challenge was making sure the map didn't freeze the screen while loading, which showed me how to delay heavy tasks to run safely in the background.

H N B Hansana - Building the save feature for the account settings showed me that we have to protect the system from bad user input. I learned how to set up strict validation, like enforcing password lengths and catching errors if someone types letters instead of numbers for their phone number, which keeps the whole application from crashing.

W G C Isidhara - Developing the main window and sidebar navigation taught me a lot about managing application state and user permissions. I learned how to securely check a user's role on startup and dynamically hide or show features.

P D Weerathunga - Working on loading the account settings taught me how data moves from a secure database directly to the user's screen. I learned how important it is to pull existing information efficiently so the user doesn't have to retype their details

N Nimjaya - Building the account request form taught me how to safely collect user input and catch errors before they ever reach the database. I also learned a lot about how our code talks to the cloud, especially how to match our local data types perfectly with Supabase

P M L Hewawasam - Building the heritage site form taught me how to be efficient with my code by reusing the exact same window for both adding new sites and updating existing ones.

6. Future Work

Further Development

While VMS effectively utilizes email for communication, emails are not always checked in real-time by tourists on the move. A critical future development would be the integration of a short messaging service (SMS, to dispatch instant, high-priority emergency alerts directly to mobile devices.

Additionally, the system could be expanded to include Multilingual Support. Given the diversity of tourists visiting Sri Lankan heritage sites, the web portal and subsequent automated emails should automatically detect the user's browser language and deliver historical information in Sinhala, Tamil, English, or other relevant languages. Finally, introducing predictive analytics into the WPF dashboard using historical data to forecast peak visitor times would greatly assist administrators in staff allocation.

Sustainable Product Changes

To ensure the long-term sustainability and compliance of VMS, two major architectural changes are necessary. First, we must implement a strict **Data Retention Policy**. Currently, visitor emails are stored indefinitely. To align with global data privacy standards (such as GDPR) and minimize our cloud storage footprint, a scheduled background check should be introduced to automatically purge visitor emails 30 days after their visit.

Secondly, to reduce idle compute costs, the desktop application's mapping and analytics features could be migrated to a fully serverless web-based dashboard (using a framework like React or Next.js). This would eliminate the need for administrators to install and update local software, allowing the system to run entirely on the cloud, scaling compute resources elastically only when required.